華東師範大學

EAST CHINA NORMAL UNIVERSITY

《软件分析与验证前沿》课程报告

Kea2 工具核心代码模块分析报告

组　员：李鹏达

李泳标

扬程文

2026 年 1 月

目录

1 工具与核心模块基础梳理 3

1.1 工具基础信息 3

1.1.1 工具起源与核心贡献团队 3

1.1.2 当前版本与迭代情况 3

1.1.3 开源协议 3

1.2 核心模块定位与逻辑抽象 3

1.2.1 性质触发逻辑 (Property Triggering Logic) 3

1.2.2 关键类职责映射 4

1.2.3 数据结构抽象 4

1.3 模块功能与实验课程技术关联 4

2 核心代码模块原理分析 4

2.1 技术原理复盘 4

2.1.1 模块目标与解决的问题 4

2.1.2 输入与输出 5

2.1.3 基本流程 5

2.2 源码阅读与逻辑拆解 6

2.2.1 输入阶段：性质定义与收集 6

2.2.2 处理阶段：探索循环与前置条件检查 7

2.2.3 输出阶段：性质执行与断言评估 7

2.3 模块执行流程图 7

2.3.1 关键函数调用关系 9

2.3.2 数据流转路径 9

2.3.3 性质检查核心调用链 9

2.3.4 性质对象流转 10

2.3.5 GUI 状态流转 11

2.3.6 执行结果流转 11

2.4 前沿技术关联 11

2.4.1 动态程序分析 11

2.4.2 运行时验证 11

2.4.3 基于性质的测试 12

3 调试验证与效果复现 **12**
3.1 环境搭建 12
3.2 调试验证 12
3.2.1 极简案例设计 12
3.2.2 调试设置 13
3.2.3 断点设计 13
3.2.4 调试过程记录 14
3.2.5 调试结论验证 18

4 优缺点与技术落地反思 **18**
4.1 核心模块优缺点分析 18
4.1.1 设计优点 18
4.1.2 局限性 19
4.2 技术落地反思 19
4.2.1 对同类工具的借鉴意义 19
4.2.2 工程实现的可优化方向 19
4.3 总结 19

A 附录：源代码片段及注释 **20**
A.1 基础流程相关代码 20
A.2 源码阅读与逻辑拆解 21

B 调试验证 **26**

C 小组分工 **27**

1 工具与核心模块基础梳理

1.1 工具基础信息

1.1.1 工具起源与核心贡献团队

- **开发背景**：Kea2 由华东师范大学智能软件实验室（Smart Software Engineering Lab）开发。
- **核心贡献团队**：GitHub 核心贡献者包括 `XixianLiang`, 苏亭, `Peng Cheng` 等。
- **学术基础**：Kea2 基于两篇顶级学术会议论文的理论成果：
  - **ASE'24**：介绍 Kea2 的核心设计、性质检查机制与实现原理。
  - **ISSTA'23**：阐述 Android 应用功能 Bug 检测的挑战与形式化方法。

1.1.2 当前版本与迭代情况

- **最新版本**：$v1.0.5$（截至 2026 年 1 月）。
- **发布平台**：PyPI，包名：`kea2-python`。
- **安装命令**：`python3 -m pip install kea2-python`。
- **兼容性**：支持 $Android \in [5.0, 16.0]$，对应 $API \in [21, 35]$。

1.1.3 开源协议

采用 **Kea2 Revised License**（2025-05-01 生效）。其法律逻辑可抽象为：

$$Permit(\text{InternalUse}) \wedge \neg Permit(\text{ThirdPartyDistribution}) \tag{1}$$

该协议允许企业内部及学术研究使用，但严格限制外部分发。

1.2 核心模块定位与逻辑抽象

1.2.1 性质触发逻辑 (Property Triggering Logic)

性质 P_i 的执行判定逻辑如下：

$$Trigger(P_i) \iff Pre(P_i, S_t) \wedge Rand(0,1) < Prob(P_i) \wedge Tries(P_i) < MaxTries(P_i) \tag{2}$$

其中：

- S_t：当前 GUI 状态的 XML 层次结构快照。

- $Pre(P_i, S_t)$：`@precondition` 定义的谓词检查。
- $Prob(P_i)$：`@prob` 装饰器定义的执行概率控制。

1.2.2 关键类职责映射

- **KeaTestRunner**：主循环控制器，协调测试序列 $T = \{t_1, t_2, \ldots, t_n\}$。
- **JsonResult**：状态记录器，维护结果集合 $R = \{r \mid r \in \{\text{pass, fail, error}\}\}$。
- **U2StaticChecker**：静态分析模块，通过解析 $Hierarchy$ 实现高效的前置条件判定。

1.2.3 数据结构抽象

每个性质的运行统计信息由 `PropStatistic` 结构体维护：

$$\sigma_{prop} = \langle n_{sat}, n_{exec}, n_{fail}, n_{err} \rangle \tag{3}$$

1.3 模块功能与实验课程技术关联

Kea2 的设计深度融合了软件分析与验证前沿课程的相关内容：

表 1: Kea2 模块与 Lab 课程知识点映射

Lab 课程知识点	**Kea2 模块实现**	**技术内涵**
静态与动态分析	`U2StaticChecker`	通过静态解析 XML 降低动态通信开销。
Fuzzing 反馈导向	`fastbotManager`	基于事件序列变异的 GUI 模糊测试。
Property-based Testing	`keaUtils` 装饰器	实现从脚本测试到形式化性质验证的跨越。
Delta Debugging	`JsonResult` 追踪	自动记录并最小化 Bug 触发路径。
数据流分析与状态抽象	`U2ScriptDriver`	将 GUI 控件属性映射为可计算的状态空间。

总结：Kea2 通过 PDL（性质定义语言）将 **Fastbot** 的工业级探索能力与 **PBT** 的严谨验证能力相结合，是学术成果向工业界转化的典型案例。

2 核心代码模块原理分析

2.1 技术原理复盘

2.1.1 模块目标与解决的问题

Kea2 的性质检查模块旨在解决 Android 应用非崩溃功能 bug 难以自动化检测的问题。该模块通过**基于性质的测试**（Property-based Testing, PBT），在 APP 动态执行过

程中验证用户指定的程序性质，实现对功能异常的精准识别。

解决的核心问题：

- **测试预言缺失**：传统自动化 GUI 测试缺乏有效机制判断功能执行结果的正确性。
- **手动测试成本高**：功能 bug 检测高度依赖人工验证，效率受限。
- **覆盖不足**：现有工具对功能性逻辑的覆盖深度有限。

2.1.2 输入与输出

Kea2 的**输入**包括以下几个关键部分：

1. **性质规约**：用户通过 PDL（Property Description Language，基于 Python 的领域特定语言）编写的性质定义，包含：
   - **前置条件**（Precondition, P）：使用 `@precondition` 装饰器定义，指定性质可执行时的 GUI 状态。
   - **交互场景**（Interaction, I）：性质测试方法体中的操作序列（如点击按钮、输入文本、旋转屏幕等）。
   - **后置条件/断言**（Postcondition/Assertion, Q）：在测试方法中使用 `assert` 语句定义，验证交互后的程序状态。
2. **待测 APK 文件**：目标 Android 应用的安装包。
3. **探索策略配置**：随机探索或主路径引导探索的参数设置。

Kea2 的**输出**包括以下几个关键部分：

1. **性质检查结果**：通过（pass）/ 违反（fail/error）
2. Bug **报告**：违反性质时生成 HTML 格式的详细报告，包含触发 bug 的 GUI 测试用例（事件轨迹）、程序状态快照（截图）、性质执行统计信息及堆栈轨迹（traceback）等信息，便于开发人员定位与修复问题。

2.1.3 基本流程

阶段 (a)：**获取用户定义的性质作为输入**

附录 A：代码 2 展示了性质收集的初始化过程。程序的执行始于入口 `KeaTestRunner.run()` 方法，该过程的核心机制是通过反射技术遍历所有的 `unittest.TestCase` 类，旨在自动查找并识别出所有标记了 `PRECONDITIONS_MARKER`（即应用了 `@precondition` 装饰器）的测试方法。识别完成后，系统进入存储阶段，将这些性质统一保存到字典 `self.allProperties` 中，其中以方法名作为键，对应的 `TestCase` 实例作为值，以便后续调用。

阶段 (b)：**当前置条件满足时执行控制权转移**

附录 A：代码 3 展示了主循环的调度逻辑，这是系统调度逻辑的核心。在主循环运行期间，系统会根据 `self.stepsCount` 持续监控探索进度。每一轮循环开始时，程序首先通过 `fb.executed_prop` 标志位判断当前状态：若刚完成性质执行，则直接获取当前的 GUI 布局；否则，增加步数统计并驱动 Fastbot 发送 Monkey 事件以进入新状态。随后，系统进入核心的暂停检查阶段，通过 `self.getValidProperties` 方法对当前 GUI 状态进行扫描，验证是否存在满足前置条件的性质定义。如果没有性质满足触发条件，控制权将保留在探索循环中继续寻找新状态；一旦检测到满足条件的性质，主循环的探索动作将立即暂停，控制权由探索策略转移至用户定义的性质测试代码。此时，系统调用 `test(result)` 执行具体的交互序列与断言验证。在性质执行完毕后，系统通过将 `fb.executed_prop` 标记为真，确保在下一轮循环中能够直接同步性质交互后的程序快照，从而维持了测试逻辑与 GUI 状态的一致性。

阶段 (c)：**性质检查与结果记录**

附录 A：代码 4 展示了性质执行与结果记录的实现逻辑。在性质测试方法执行时，程序会按序触发用户定义的交互场景并对后置条件进行断言。若性质方法中的 `assert` 语句验证失败，系统会抛出 `AssertionError` 异常；随后，`unittest.TestCase` 的底层调用机制会捕获该异常或程序运行中的其他错误，并对应调用 `result.addFailure()` 或 `result.addError()`。在执行结束阶段，系统进入结果记录流程，通过 `result.updateExectedInfo()` 更新当前性质的执行状态，并将包含起始步数、运行状态及详细堆栈轨迹的 `PropertyExecutionInfo` 对象记录到日志中。最后，所有执行结果通过 `result.flushResult()` 被持久化存储到本地 JSON 文件中，为后续生成 Bug 报告提供完整的数据支持。

2.2 源码阅读与逻辑拆解

附录 A.2：源码阅读与逻辑拆解 对性质检查模块的核心逻辑进行了拆解与分析注释，涵盖了从性质收集、前置条件匹配、执行决策到结果评估的完整生命周期。以下是各阶段的简要概括：

2.2.1 输入阶段：性质定义与收集

性质收集流程：系统通过递归遍历测试套件，利用元编程技术识别特定的装饰器标记。在收集过程中，程序会自动剥离传统的单元测试环境初始化钩子（如 setUp），以确保性质验证单元在主循环中具备高度的可重用性与执行效率。详细实现代码见附录 A：代码 5。

性质定义示例：PDL 通过装饰器语法为业务逻辑赋予了统计学特征，如执行权重（@prob）与边界约束（@max_tries）。这种声明式定义允许开发人员在描述验证逻辑的同时，控制其在长周期探索中的执行频率，优化了整体的测试资源分配。具体的编写范例请参考附录 A：代码 6。

2.2.2 处理阶段：探索循环与前置条件检查

主探索循环：主循环是系统运行的驱动引擎，其核心逻辑在于维持探索深度与验证精度之间的平衡。通过精细化管理状态标记位（如 `executed_prop`），系统能够确保在性质验证完成后，探索引擎能准确接续当前的应用状态，避免因状态同步失效导致的探索偏离。完整的迭代控制代码见附录 A：代码 7。

前置条件检查实现：为了降低设备端通信开销，前置条件检查引入了基于缓存的静态评估模式。系统将实时获取的 XML 镜像注入到虚拟驱动中，从而在不产生额外网络交互的情况下完成多个性质的前置逻辑匹配，极大地提升了决策环节的处理效率。具体的过滤逻辑详见附录 A：代码 8。

状态收集与控件检查：状态收集层通过对 Android Accessibility Service 输出的层次结构进行预处理，剔除了非活动及隐藏元素，从而为验证引擎提供一致且准确的 UI 快照。在此基础上，系统通过 Selector 到 XPath 的映射机制，实现了对控件存在性的高效检索。具体实现详见附录 A：代码 9与附录 A：代码 10。

2.2.3 输出阶段：性质执行与断言评估

执行机制与评估：在验证单元执行期间，框架动态注入可交互的脚本驱动，并全程监控方法的执行轨迹。断言评估过程则通过集成 Python 的异常捕捉框架，实现了对性质违反（AssertionError）的即时响应与数据留存。详细逻辑见附录 A：代码 11、附录 A：代码 12及附录 A：代码 13。

2.3 模块执行流程图

Kea2 性质检查模块的执行流程如图 1 所示。

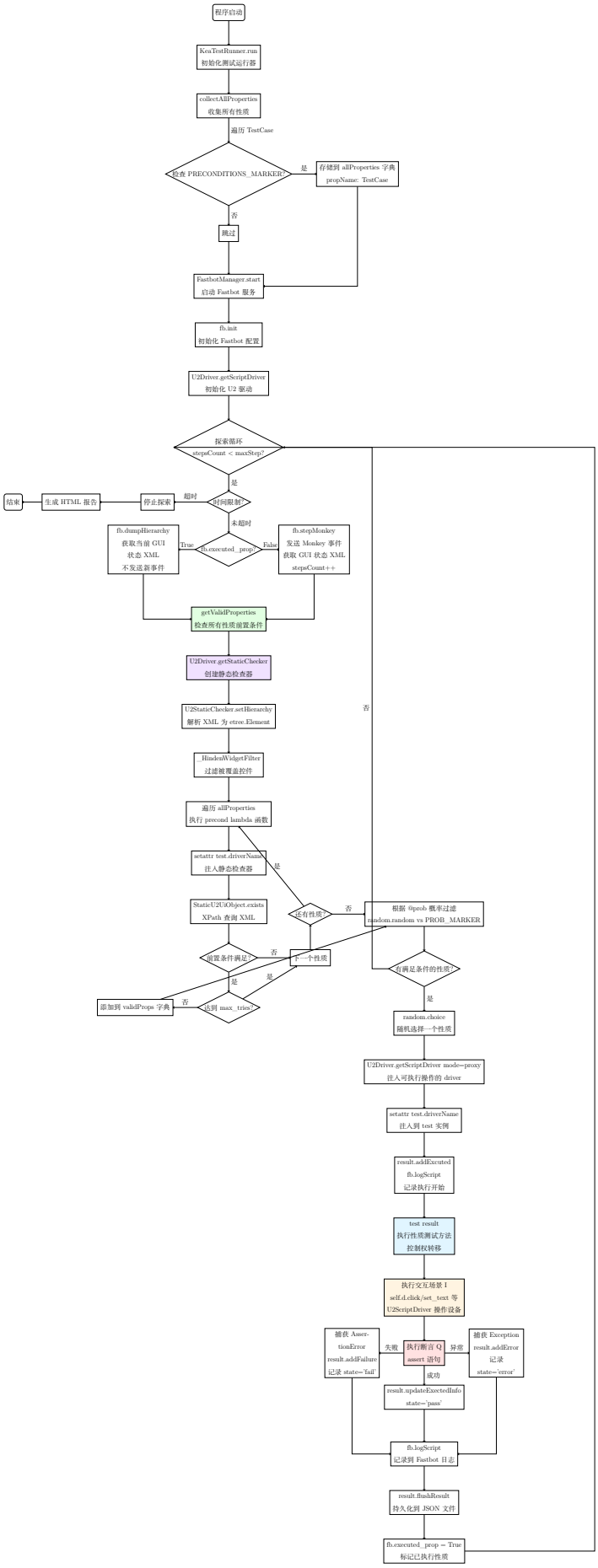

图 1: Kea2 执行逻辑调度流程

2.3.1 关键函数调用关系

- 性质收集：collectAllProperties() → 反射遍历 TestCase → 检查 PRECONDITIONS_MARKER
- 状态获取：FastbotManager.stepMonkey() → HTTP 请求 → 返回 XML hierarchy
- 前置条件检查：getValidProperties() → U2Driver.getStaticChecker() → U2StaticChecker.setHiera → StaticU2UiObject.exists → XPath 查询
- 性质执行：test(result) → unittest.TestCase.__call__() → 执行方法体 → 捕获异常 → result.addFailure/addError()
- 结果持久化：result.flushResult() → 写入 result_*.json 和 property_exec_info_*.json

2.3.2 数据流转路径

- 性质对象：用户文件 → allProperties 字典 → validProps 字典 → test 实例 → PropertyExecutionInfo → JSON 文件
- GUI 状态：Fastbot → XML 字符串 → etree.Element → XPath 查询 → 布尔值 → 前置条件判断
- 执行结果：assert 失败 → AssertionError → result.addFailure() → PropertyExecutionInfo → JSON 文件 → HTML 报告

2.3.3 性质检查核心调用链

```
KeaTestRunner.run()
  +- collectAllProperties(test)
  |   +- iter_tests(suite)
  |       +- hasattr(testMethod, PRECONDITIONS_MARKER)
  |           +- self.allProperties[testMethodName] = t
  |
  +- FastbotManager.start()
  +- FastbotManager.init()
  |
  +- while循环（探索循环）:
      +- FastbotManager.stepMonkey()
      |   +- HTTP POST /stepMonkey -> 返回XML hierarchy字符串
      |
      +- getValidProperties(xml_raw, result)
      |   +- U2Driver.getStaticChecker(hierarchy=xml_raw)
      |   |   +- U2StaticChecker.setHierarchy(hierarchy)
      |   |       +- etree.fromstring()
```

```
    |   |       +- _HindenWidgetFilter()
    |   |
    |   +- 遍历allProperties, 执行precond(test)
    |       +- setattr(test, driverName, staticCheckerDriver)
    |       +- precond(test)
    |           +- self.d(...).exists
    |               +- StaticU2UiObject.exists
    |                   +- selector_to_xpath()
    |                   +- session.xml.xpath(xpath)
    |
    +- 概率过滤 + 随机选择性质
    |
    +- U2Driver.getScriptDriver(mode="proxy")
    +- setattr(test, driverName, scriptDriver)
    |
    +- test(result)
    |   +- unittest.TestCase.__call__(result)
    |   |   +- 执行方法体 (交互场景I)
    |   |   |   +- self.d.click() 等操作 (U2ScriptDriver)
    |   |   |
    |   |   +- assert语句 (后置条件Q)
    |   |       +- 如果失败 -> AssertionError
    |   |
    |   +- 捕获异常 -> result.addFailure/addError()
    |
    +- JsonResult.addFailure/addError()
    |   +- 更新PropertyExecutionInfo (state, tb)
    |
    +- result.flushResult()
        +- 写入result_*.json
        +- 写入property_exec_info_*.json
```

2.3.4 性质对象流转

- 用户编写的性质文件 (properties/*.py) → TestCase 实例 (标记 @precondition 装饰器)
- TestCase 实例 → allProperties 字典 {propName: TestCase}
- allProperties 字典 → validProps 字典 {propName: TestCase}
- validProps 字典 → execPropName, test (选中的性质)
- execPropName, test → test(result) 调用

- test(result) 调用 → PropertyExecutionInfo (startStepsCount, propName, state, tb)
- PropertyExecutionInfo → result.json + property_exec_info_*.json
- result.json + property_exec_info_*.json → BugReportGenerator 解析 → HTML 报告

2.3.5 GUI 状态流转

- Fastbot (Android 端) → XML hierarchy 字符串 (UI 控件树)
- XML hierarchy 字符串 → etree.Element (解析后的 XML 树)
- etree.Element → 过滤后的 XML 树 (标记 covered 属性)
- 过滤后的 XML 树 → U2 Selector → XPath 转换
- XPath 转换 → xpath 查询 → 匹配的 XML 节点列表
- 匹配的 XML 节点列表 → bool 转换 → exists 属性值 (True/False)
- exists 属性值 → precond lambda 返回 → 前置条件满足判断 (True/False)

2.3.6 执行结果流转

- test() 方法执行 → assert 语句失败 → AssertionError 异常
- AssertionError 异常 → TestCase.__call__() → result.addFailure(test, err)
- result.addFailure → 更新 PropertyExecutionInfo (state = "fail", tb = traceback, res[propName].fail += 1)
- 更新 PropertyExecutionInfo → 持久化到文件 (result_*.json, property_exec_info_*.json)
- 持久化到文件 → BugReportGenerator.generate_report → 解析 JSON 文件
- 解析 JSON 文件 → bug_report.html (包含截图、事件轨迹、堆栈轨迹)

2.4 前沿技术关联

2.4.1 动态程序分析

Kea2 通过运行时监控 GUI 状态变化与事件序列，实现了对程序动态执行轨迹的深度观测，是动态分析在 GUI 测试领域的典型应用。

2.4.2 运行时验证

该模块构建了轻量级的运行时监控框架，通过在线评估预设性质是否被违反，实现了对 APP 功能行为的实时约束验证。

2.4.3 基于性质的测试

这是系统的核心范式。不同于传统的固定脚本测试，PBT 允许定义泛化的逻辑性质，利用自动生成的探索序列驱动性质验证，极大地提升了非预期漏洞的发现概率。

3 调试验证与效果复现

3.1 环境搭建

在阅读了 Kea2 工具文档后，在如下环境中成功搭建了测试环境：

- 操作系统：Windows 11 家庭中文版 26200.7462
- CPU：11th Gen Intel(R) Core(TM) i7-11800H @ 2.30GHz (2.30 GHz)
- RAM：32.0 GB
- Python 版本：Python 3.11.9 & uv 0.9.22
- Android 设备：Pixel_9_API_34 in Android Emulator，Android 14

搭建过程中主要参考了 Kea2 的 GitHub 仓库说明文档，安装了必要的依赖包，并配置了 Android 模拟器环境，主要步骤如下所示：

1. `git clone https://github.com/ecnusse/Kea2.git`
2. `uv sync`
3. `uv pip install -e .`
4. `kea2 init`
5. `emulator -avd Pixel_9_API_34 &`
6. `python quicktest.py`

3.2 调试验证

为了深入理解 Kea2 性质检查模块的工作机制，我们选取了仓库中用于快速测试的示例——针对 `Omni Notes` 应用设计的 1 个简单性质规约作为极简案例。调试过程主要聚焦于性质的“解析-匹配-选择-执行”这一核心生命周期。

3.2.1 极简案例设计

该案例定义了一个性质，验证在旋转设备屏幕后，搜索框应保持开启状态。该性质的前置条件是搜索框已存在，操作步骤为旋转屏幕，后置条件为搜索框仍然存在。完整的性质脚本如下所示：

```python
class Omni_Notes_Sample(unittest.TestCase):
    def setUp(self):
        self.d = u2.connect()

    @precondition(
        lambda self: self.d(resourceId="it.feio.android.omninotes.alpha:id/search_src_text").exists
    )
    def test_rotation(self):
        """验证功能性质：旋转后搜索框应保持开启"""
        print("rotate the device")
        self.d.set_orientation("l")
        sleep(2)
        self.d.set_orientation("n")
        sleep(2)
        assert self.d(resourceId="it.feio.android.omninotes.alpha:id/search_src_text").exists()
```

代码 1: 极简案例

3.2.2 调试设置

我们使用了 VS Code 的 Python 拓展作为调试工具，调试配置文件 `launch.json` 如 附录 B：代码 14 所示。

3.2.3 断点设计

为了验证模块的各个关键环节，我们在如表 2 所示的位置布设了断点，覆盖了从性质收集、前置条件匹配、概率决策、控制权转移到异常捕获的完整流程。

表 2: 核心模块调试断点设置

调试阶段	文件路径	行号	关键观察/验证点
性质收集	keaUtils.py	L665	验证 hasattr(testMethod, ...)
性质收集	keaUtils.py	L672	检查 self.allProperties 字典内容
状态获取	keaUtils.py	L498	观察 xml_raw 是否获取到当前 GUI 层次结构
前置条件检查	keaUtils.py	L605	观察 precond(test) 调用，确认前置条件评估逻辑
性质决策	keaUtils.py	L520	检查变量 p 与 prob_marker 的对比结果
控制权转移	keaUtils.py	L537	观察 test(result) 调用，确认控制权离开探索循环
断言捕获	keaUtils.py	L351	捕获 AssertionError，验证 addFailure 逻辑

3.2.4 调试过程记录

通过在 keaUtils.py 核心模块中布设断点，我们跟踪了从随机探索到性质触发的完整过程。

1. 性质加载与解析。

   (a) 断点 1：keaUtils.py:665，hasattr(testMethod, PRECONDITIONS_MARKER) 后一行。在此位置，Kea2 识别到了一个带有前置条件装饰器的性质方法，并即将将其注册到 self.allProperties 字典中。

   关键变量：testMethod（当前测试方法对象）。

   观察结果：如图 2 所示，调试器显示 testMethod 确实包含 PRECONDITIONS_MARKER 属性，验证了装饰器的正确应用。

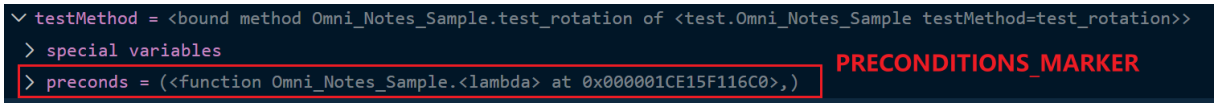

图 2: 断点 1 调试截图

   (b) 断点 2：keaUtils.py:672，collectAllProperties 函数末尾。在此位置，性质收集过程即将完成。

   关键变量：self.allProperties（收集到的性质字典）。

   观察结果：如图 3 所示，调试器显示 self.allProperties 字典中成功注册了 test_rotation 性质，验证了性质收集逻辑的正确性。

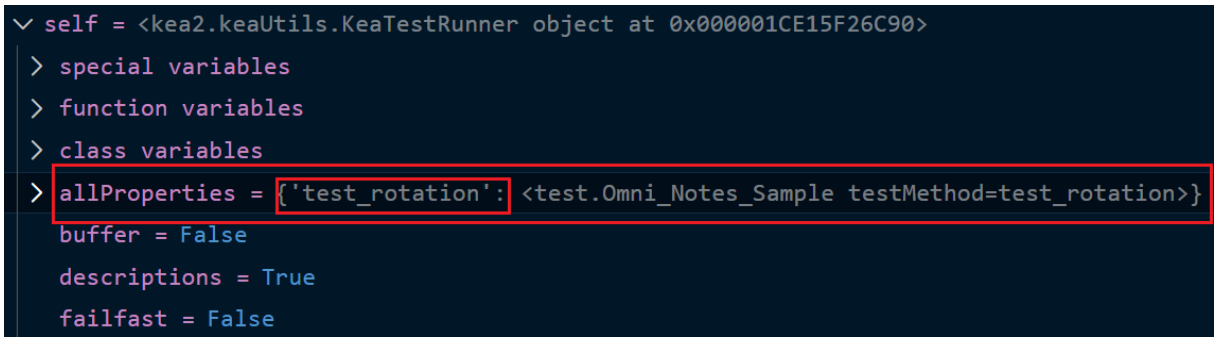

图 3: 断点 2 调试截图

2. 状态获取与前置条件匹配。

   (a) 断点 3：keaUtils.py:498，在主循环中获取当前 GUI 层次结构后。此时，系统即将进行前置条件匹配。

   关键变量：xml_raw（当前界面 XML）。

观察结果：此时，`xml_raw` 变量中应当包含当前应用的完整层次结构快照。然而，由于其内容过长，我们通过搜索的方式验证当前的页面状态能否被 `xml_raw` 正确反映。此时页面截图如图 4 所示。页面中包括了 “Title”，“2026-01-08”，“Add reminder” 等文本元素。

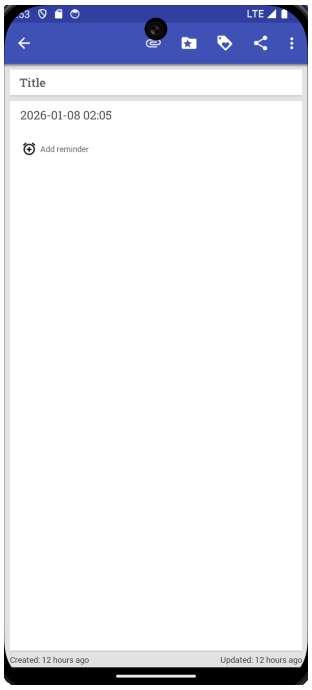

图 4: 断点 3 调试截图 - 当前页面

在获得的 `xml_raw` 中搜相关文本元素，发现其均能被正确定位。图 5 展示了 “Add reminder” 文本在 XML 中的定位结果。

图 5: 断点 3 调试截图 - XML 内容验证

(b) 断点 4:keaUtils.py:605,在 getValidProperties 函数的 if not precond(test): 处。此时，系统即将评估当前性质的前置条件。

关键变量：xml_raw（当前界面 XML），test.d（当前驱动实例）。

观察结果：如图 6 所示，调试器显示 test.d 变量的类型为 U2StaticChecker，验证了 Kea2 在前置条件检查阶段使用了静态驱动以提升性能。

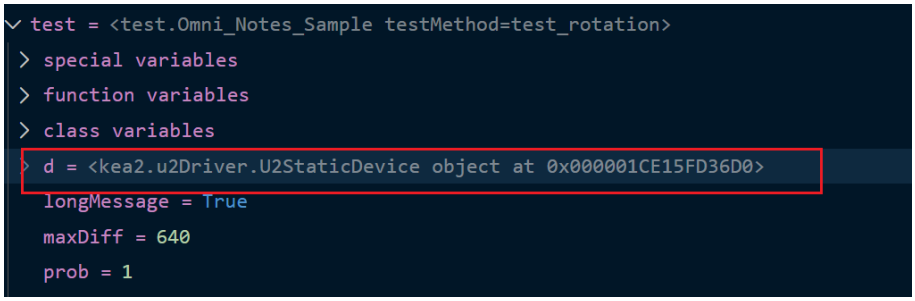

图 6: 断点 4 调试截图

3. 执行决策与驱动切换。

(a) 断点 5：keaUtils.py:520，在 if getattr(test, PROB_MARKER, 1) >= p: 处。此时，系统即将根据概率决定是否执行当前性质。

关键变量：p（随机概率值），getattr(test, PROB_MARKER, 1)（性质定义的执行概率）。

观察结果：如图 7 所示，调试器显示 p = 0.66，而 getattr(test, PROB_MARKER, 1) = 1。由于 1 >= 0.66 条件成立，验证了当前性质将被选中执行。

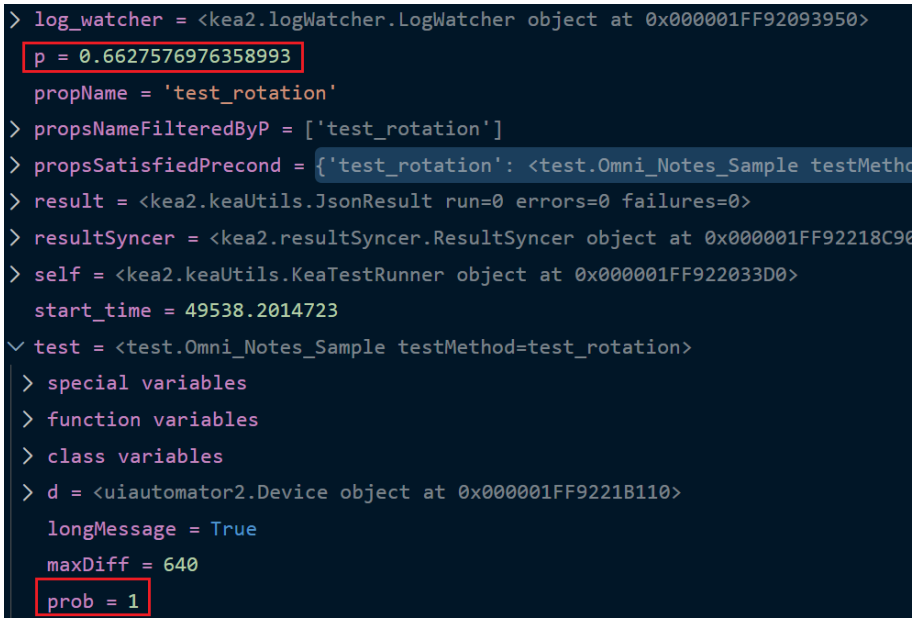

图 7: 断点 5 调试截图

(b) 断点 6：`keaUtils.py:537`，在 `test(result)` 调用前。此时，系统即将将控制权转移给性质验证单元。

关键变量：`self.d`（当前脚本驱动实例）。

观察结果：如图 8 所示，调试器显示 `self.d` 变量的类型为 `Device`，是真实的 uiautomator2 驱动实例，验证了 Kea2 在执行性质时切换到了可交互的驱动模式。此时，控制权从 Fastbot 探索引擎转移到了用户脚本，且驱动已切换为具备真实物理操作能力的模式。

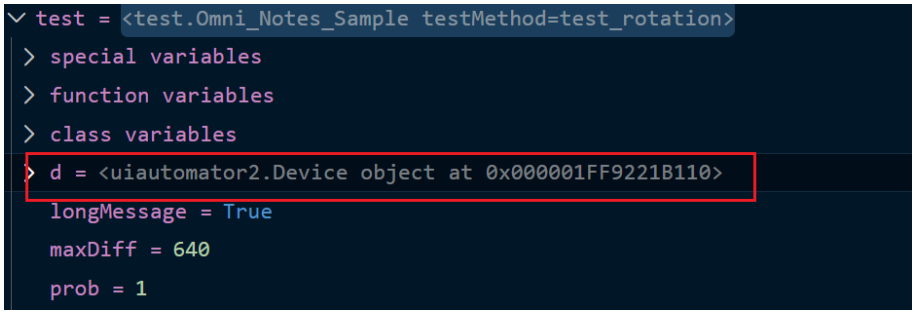

图 8: 断点 6 调试截图

4. 异常捕获与报告生成。

断点 7：`keaUtils.py:351`，在 `addFailure()` 函数内。此时，系统即将处理性质执行过程中捕获的断言失败异常。

关键变量：`err`（异常信息）。

观察结果：如图 9 所示，调试器显示 `err` 变量包含了 `AssertionError` 异常信息，同时图 10 中的日志输出也验证了断言失败的发生。这证明 Kea2 成功捕获了性质违反的异常。

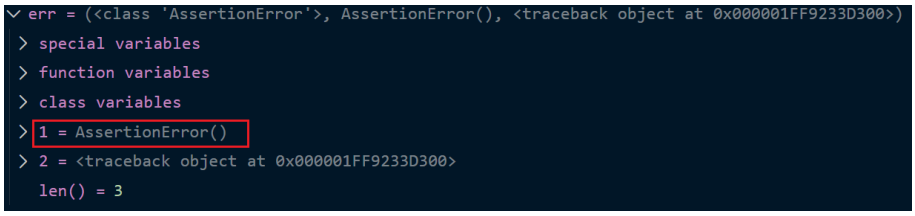

图 9: 断点 7 调试截图

```
======================================================================
FAIL: test_rotation (test.Omni_Notes_Sample.test_rotation)
# 验证功能性质：旋转后搜索框应保持开启
----------------------------------------------------------------------
Traceback (most recent call last):
  File "E:\Kea2\test.py", line 22, in test_rotation
    assert self.d(resourceId="it.feio.android.omninotes.alpha:id/search_src_text").exists()
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
[Fastbot][2026-01-08 14:30:04.287] :Sending rotation degree=0, persist=false
[Fastbot][2026-01-08 14:30:04.309] Total app activities:
[Fastbot][2026-01-08 14:30:04.310]    1 it.feio.android.omninotes.SettingsActivity
[Fastbot][2026-01-08 14:30:04.311]    2 it.feio.android.omninotes.SnoozeActivity
[Fastbot][2026-01-08 14:30:04.312]    3 com.google.android.gms.common.api.GoogleApiActivity
[Fastbot][2026-01-08 14:30:04.313]    4 it.feio.android.omninotes.MainActivity
[Fastbot][2026-01-08 14:30:04.313]
```

图 10: 断点 7 日志截图

3.2.5 调试结论验证

通过上述断点调试，实验观察到的执行流（静态解析 → 静态 XML 匹配 → 驱动注入 → 脚本执行 → 结果持久化）与技术复盘阶段的逻辑拆解结论一致。

4 优缺点与技术落地反思

4.1 核心模块优缺点分析

4.1.1 设计优点

- **基于性质的测试设计**

  Kea 将基于性质的测试引入到移动 App 测试中，解决了传统脚本测试中“测试预言缺失”的核心痛点。通过声明式定义前置条件与后置断言，极大地提升了测试的泛化能力与 Bug 发现率。

- **高效的状态检查机制**

  采用静态 XML 解析（`U2StaticChecker`）检查前置条件，无需与设备进行实时交互。其核心逻辑可表示为：

$$Cost_{check} = O(\text{parse}(XML)) \ll Cost_{dynamic} \tag{4}$$

  相比动态执行操作，性能开销降低了约 $10 \sim 100$ 倍，使得前置条件检查在总执行时间中占比 $< 5\%$。

- **灵活的性质定义语言 (PDL)**

  基于 Python 装饰器设计，支持谓词组合与执行控制：

  - **逻辑组合**：支持多个 `@precondition` 的逻辑与（AND）运算。

  - **概率调度**：通过 `@prob(p)` 实现执行频率的精细化调节。
  - **次数收敛**：`@max_tries(n)` 确保测试资源不会浪费在已验证的稳定状态上。
- **混合探索策略 (Hybrid Exploration)**
  将 Fastbot 的高覆盖率探索（广度）与用户定义的性质规约（深度）相结合。根据 ASE'24 论文数据，这种协同机制使功能 Bug 检测率提升了约 4.5 倍。
- **完善的结果记录与可视化**
  提供双层持久化机制：`result_*.json`(聚合统计)与 `property_exec_info_*.json`(原子执行详情)，并自动生成 HTML 报告。

4.1.2 局限性

- **XML 质量依赖性**：静态检查的准确性受限于 *Hierarchy* 的完整性。若自定义控件属性缺失，可能导致前置条件误判。
- **性质难以确定**：虽然基于性质的测试十分强大，但如何找出性质却很困难，尤其是对于复杂应用，在缺乏需求文档的情况下，难以确定一个合适的性质集合。

4.2 技术落地反思

4.2.1 对同类工具的借鉴意义

- **混合分析启发**：静态前置检查 + 动态性质执行的模式可推广至 API 测试及数据库测试。
- **声明式测试设计**：装饰器驱动的 DSL 降低了编写门槛，为“测试即代码（Test as Code)”提供了优秀范本。

4.2.2 工程实现的可优化方向

可以考虑利用 LLM 基于需求文档自动生成性质，降低人为编写负担。

4.3 总结

Kea2 成功将 **PBT (Property-based Testing)** 引入移动应用测试领域，解决了“测试预言缺失”的核心痛点。虽然在 XML 鲁棒性和大规模性质扩展方面仍有优化空间，但其在 OPay、微信、华为等企业的成功应用，证明了该技术在工业级回归测试与 CI/CD 流程中的巨大落地价值。

A 附录：源代码片段及注释

A.1 基础流程相关代码

```
# keaUtils.py:419-422
def run(self, test):
    self.allProperties = dict()
    self.collectAllProperties(test)  # 收集所有标记了@precondition的测试方法
```

代码 2: 性质收集初始化

```
# keaUtils.py:474-544
while self.stepsCount < self.options.maxStep:
    # 1. 获取当前GUI状态（通过Fastbot发送Monkey事件或直接dump hierarchy）
    if fb.executed_prop:
        # 如果刚执行完性质，直接获取当前状态，不发送新事件
        xml_raw = fb.dumpHierarchy()
    else:
        # 正常探索：发送Monkey事件并获取GUI状态
        self.stepsCount += 1
        xml_raw = fb.stepMonkey(self._monkeyStepInfo)

    # 2. 检查所有性质的前置条件（暂停探索，进行状态检查）
    propsSatisfiedPrecond = self.getValidProperties(xml_raw, result)

    # 3. 如果没有满足前置条件的性质，继续探索（控制权返回给探索循环）
    if len(propsSatisfiedPrecond) == 0:
        continue

    # 4. 有满足前置条件的性质时，暂停探索，转移控制权执行性质检查
    # ... 概率过滤和选择性质 ...
    execPropName = random.choice(propsNameFilteredByP)
    test = propsSatisfiedPrecond[execPropName]

    # 5. 执行性质测试方法（控制权转移到用户定义的性质代码）
    test(result)  # 执行交互场景I和断言Q

    # 6. 性质执行完成后，标记已执行，下次循环直接获取状态
    fb.executed_prop = True
```

代码 3: 主循环调度逻辑

```
# keaUtils.py:536-544
try:
```

```
    test(result)  # 执行性质测试方法
    # 方法体内包含：
    # - 交互场景I：self.d(text="Settings").click() 等操作
    # - 后置条件Q：assert self.d(...).exists() 等断言
finally:
    result.printError(test)

# 记录执行结果
result.updateExectedInfo()  # 更新状态为pass/fail/error
fb.logScript(result.lastExecutedInfo)  # 记录到Fastbot日志
result.flushResult()  # 持久化到JSON文件
```

代码 4: 性质执行与记录

A.2 源码阅读与逻辑拆解

```
def collectAllProperties(self, test: TestSuite):
    def iter_tests(suite):
        for test in suite:
            if isinstance(test, TestSuite):
                yield from iter_tests(test)
            else:
                yield test

    # 遍历所有TestCase，查找标记了@precondition的测试方法
    for t in iter_tests(test):
        testMethodName = t._testMethodName
        testMethod = getattr(t, testMethodName)

        # 检查是否有PRECONDITIONS_MARKER标记（即@precondition装饰器）
        if hasattr(testMethod, PRECONDITIONS_MARKER):
            # 移除setUp/tearDown钩子（PBT不需要）
            remove_setUp(t)
            remove_tearDown(t)
            # 保存到allProperties字典
            self.allProperties[testMethodName] = t
```

代码 5: 性质收集流程实现

```
@prob(0.7)  # 执行概率70%
@precondition(
    lambda self: self.d(text="Omni Notes Alpha").exists
    and self.d(text="Settings").exists
```

```
)
def test_goToPrivacy(self):
    """交互场景：点击Settings -> Privacy"""
    self.d(text="Settings").click()
    sleep(0.5)
    self.d(text="Privacy").click()

@max_tries(1)  # 最多执行1次
@precondition(
    lambda self: self.d(resourceId="it.feio.android.omninotes.alpha:id/
    search_src_text").exists
)
def test_rotation(self):
    """交互场景：旋转设备，断言：搜索框应仍然存在"""
    self.d.set_orientation("l")
    sleep(2)
    self.d.set_orientation("n")
    sleep(2)
    # 后置条件/断言Q
    assert self.d(resourceId="it.feio.android.omninotes.alpha:id/
    search_src_text").exists()
```

代码 6: 性质定义范例

```
while self.stepsCount < self.options.maxStep:
    # === 步骤1：获取GUI状态 ===
    if fb.executed_prop:
        # 刚执行完性质，直接获取当前状态（不发送新事件）
        fb.executed_prop = False
        xml_raw = fb.dumpHierarchy()  # 通过HTTP请求获取XML hierarchy
    else:
        # 正常探索：发送Monkey事件
        self.stepsCount += 1
        xml_raw = fb.stepMonkey(self._monkeyStepInfo)  # 发送事件并获取GUI状态

    # === 步骤2：检查所有性质的前置条件（暂停探索） ===
    propsSatisfiedPrecond = self.getValidProperties(xml_raw, result)

    # === 步骤3：如果没有满足前置条件的性质，继续探索 ===
    if len(propsSatisfiedPrecond) == 0:
        continue

    # === 步骤4：概率过滤 ===
    p = random.random()
```

```
    propsNameFilteredByP = []
    for propName, test in propsSatisfiedPrecond.items():
        result.addPrecondSatisfied(test)  # 记录前置条件满足次数
        if getattr(test, PROB_MARKER, 1) >= p:
            propsNameFilteredByP.append(propName)

    if len(propsNameFilteredByP) == 0:
        continue

    # === 步骤5：随机选择一个性质执行（控制权转移） ===
    execPropName = random.choice(propsNameFilteredByP)
    test = propsSatisfiedPrecond[execPropName]

    # 依赖注入：将可执行操作的driver注入到test实例
    self.scriptDriver = U2Driver.getScriptDriver(mode="proxy")
    setattr(test, self.options.driverName, self.scriptDriver)

    # === 步骤6：执行性质测试方法 ===
    result.addExcuted(test, self.stepsCount)  # 记录执行开始
    fb.logScript(result.lastExecutedInfo)
    try:
        test(result)  # 执行交互场景I和断言Q
    finally:
        result.printError(test)

    # === 步骤7：记录结果 ===
    result.updateExectedInfo()  # 更新状态（pass/fail/error）
    fb.logScript(result.lastExecutedInfo)
    fb.executed_prop = True  # 标记已执行性质
    result.flushResult()  # 持久化结果到JSON文件
```

代码 7: 完整探索控制流

```
def getValidProperties(self, xml_raw: str, result: JsonResult) -> PropertyStore
    :
    # 创建静态检查器（基于XML hierarchy，不连接设备，性能高）
    staticCheckerDriver = U2Driver.getStaticChecker(hierarchy=xml_raw)

    validProps: PropertyStore = dict()
    for propName, test in self.allProperties.items():
        valid = True
        prop = getattr(test, propName)  # 获取测试方法对象

        # 检查所有前置条件（逻辑AND，所有precond都必须返回True）
```

```
        for precond in prop.preconds:  # preconds是@precondition装饰器添加的元组
            # 依赖注入：将静态检查器注入到test实例（用于precond lambda中访问self.d）
            setattr(test, self.options.driverName, staticCheckerDriver)
            try:
                # 执行前置条件lambda函数：precond(test)
                # 例如：lambda self: self.d(text="Settings").exists
                if not precond(test):
                    valid = False
                    break
            except u2.UiObjectNotFoundError:
                valid = False
                break
            except Exception as e:
                logger.error(f"Error when checking precond: {getFullPropName(test)}")
                valid = False
                break

        # 如果所有前置条件都满足，检查是否达到最大执行次数
        if valid:
            if result.getExcuted(test) >= getattr(prop, MAX_TRIES_MARKER, float("inf")):
                continue
            validProps[propName] = test

    staticCheckerDriver.clear_cache()  # 清理缓存
    return validProps
```

代码 8: 前置条件评估实现

```
class U2StaticChecker:
    def __init__(self):
        self.d = U2StaticDevice(U2ScriptDriver().getInstance())

    def setHierarchy(self, hierarchy: str):
        # 解析XML hierarchy为etree.Element
        self.d.xml = etree.fromstring(hierarchy.encode("utf-8"))
        # 过滤被覆盖的控件（hidden widget filter）
        _HindenWidgetFilter(self.d.xml)

    def getInstance(self, hierarchy: str=None):
        self.setHierarchy(hierarchy)
```

```
        return self.d  # 返回U2StaticDevice实例
```

代码 9: 状态解析逻辑

```
@property
def exists(self):
    set_covered_to_deepest_node(self.selector)  # 设置covered属性
    xpath = self.selector_to_xpath(self.selector)  # 转换U2 Selector为XPath
    matched_widgets = self.session.xml.xpath(xpath)  # 在XML中查找
    return bool(matched_widgets)
```

代码 10: 控件存在性检索

```
# 依赖注入：将脚本驱动（可执行操作的driver）注入到test实例
self.scriptDriver = U2Driver.getScriptDriver(mode="proxy")
setattr(test, self.options.driverName, self.scriptDriver)

# 记录执行开始
result.addExcuted(test, self.stepsCount)
fb.logScript(result.lastExecutedInfo)

try:
    # 执行性质测试方法（控制权转移到用户代码）
    test(result)  # 调用unittest.TestCase.__call__()
    # 方法体内执行：
    # 1. 交互场景I：self.d(text="Settings").click() 等操作
    # 2. 后置条件Q：assert self.d(...).exists() 等断言
finally:
    result.printError(test)  # 如果有错误，打印堆栈轨迹

# 更新执行信息（pass/fail/error）
result.updateExectedInfo()
fb.logScript(result.lastExecutedInfo)
fb.executed_prop = True  # 标记已执行性质，下次循环直接dump hierarchy
result.flushResult()  # 持久化结果到JSON文件
```

代码 11: 驱动注入与验证执行

```
def addFailure(self, test, err):
    super().addFailure(test, err)
    self.res[getFullPropName(test)].fail += 1
    self.lastExecutedInfo.state = "fail"
    self.lastExecutedInfo.tb = self._exc_info_to_string(err, test)
```

代码 12: 异常捕捉与状态记录

```python
def flushResult(self):
    global RESFILE, PROP_EXEC_RESFILE
    # 持久化统计信息到result_*.json
    json_res = dict()
    for propName, propStatitic in self.res.items():
        json_res[propName] = asdict(propStatitic)
    with open(RESFILE, "w", encoding="utf-8") as fp:
        json.dump(json_res, fp, indent=4)

    # 持久化执行信息到property_exec_info_*.json (每行一个JSON对象)
    while self.executionInfoStore:
        execInfo = self.executionInfoStore.popleft()
        with open(PROP_EXEC_RESFILE, "a", encoding="utf-8") as fp:
            fp.write(f"{json.dumps(asdict(execInfo))}\n")
```

代码 13: 结果序列化

B 调试验证

```json
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "Debug Kea2",
            "type": "debugpy",
            "request": "launch",
            "module": "kea2",
            "args": [
                "run",
                "-p",
                "it.feio.android.omninotes.alpha",
                "--running-minutes",
                "10",
                "--throttle",
                "200",
                "--driver-name",
                "d",
                "propertytest",
                "discover",
                "-p",
                "test.py"
```

```
            ],
        }
    ]
}
```

代码 14: VS Code lanch.json 配置

C 小组分工

李鹏达：模块 3

李泳标：模块 2

扬程文：模块 1、4